DESENVOLVIMENTO MOBILE · MÓDULO 11 · TRILHA KOTLIN · NÍVEL INTERMEDIÁRIO

Rede e Consumo de APIs

HTTP, REST e JSON, Retrofit, OkHttp, tratamento de erros, offline-first, imagens e segurança

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Descrever o ciclo requisição/resposta do HTTP, seus métodos e códigos de status.
- Ler a documentação de uma API REST e modelar seus dados em Kotlin.
- Consumir APIs com **Retrofit** + **kotlinx.serialization** + **OkHttp**.
- Tratar falhas de rede e representá-las no estado da tela.
- Implementar a estratégia *offline-first* com Room como fonte da verdade.
- Carregar imagens e proteger chaves e dados em trânsito.

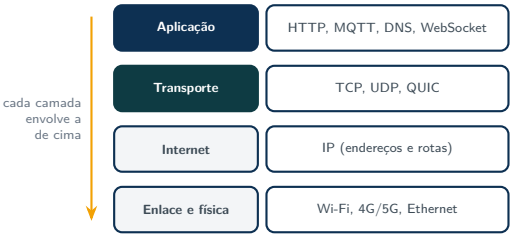
Roteiro

Neste módulo

- Fundamentos: redes de computadores
- Fundamentos de HTTP
- APIs REST e JSON
- Configuração no Android
- Retrofit e kotlinx.serialization
- Erros e estados
- Offline-first
- Imagens, alternativas e segurança
- Fechamento

Fundamentos: redes de computadores

Redes em camadas: o modelo TCP/IP



Um pouco de história

Vint Cerf e Bob Kahn publicaram o TCP/IP em 1974; a ARPANET o adotou em 1983. O modelo OSI, de sete camadas, é a referência teórica.

Protocolo

Conjunto de regras que define o formato das mensagens e a ordem da conversa entre dois computadores.

Encapsulamento

Cada camada acrescenta seu cabeçalho e entrega à de baixo: a mensagem HTTP vai dentro de um segmento TCP, que vai dentro de um pacote IP, que vai dentro de um quadro de Wi-Fi ou 4G.

Dica

O app só programa a camada de aplicação; o sistema operacional cuida do resto. Mas conhecer as de baixo explica lentidão e falhas.

Endereços, portas, NAT e DNS

Endereço IP e porta

O IP identifica a máquina (IPv4 tem 32 bits; IPv6, 128 bits). A porta identifica o programa: 443 para HTTPS, 8883 para MQTT seguro.

DNS

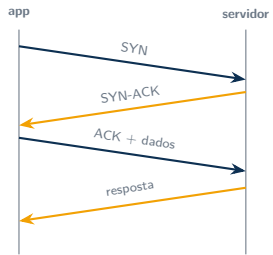
Sistema hierárquico que traduz nomes em IPs: o resolvedor consulta a raiz, depois o domínio de topo (`.dev`) e por fim o servidor responsável. As respostas ficam em cache.



Por que o servidor não "chama" o celular

O celular fica atrás de um NAT da operadora, com endereço privado: ele abre conexões, mas não recebe conexões novas. Por isso existem as notificações *push* e as conexões mantidas abertas pelo próprio app.

TCP e UDP



aperto de mão em três vias:
uma ida e volta antes dos dados

	TCP	UDP
Conexão	estabelecida antes dos dados	nenhuma: cada pacote é independente
Garantias	entrega, ordem e controle de congestionamento	nenhuma: pacotes podem se perder ou chegar fora de ordem
Custo	mais idas e voltas	mínimo
Usos	HTTP/1.1, HTTP/2, MQTT	DNS, voz, jogos, QUIC

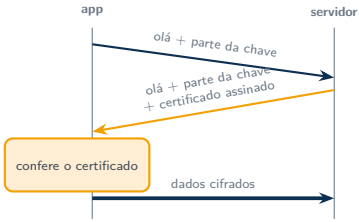
Latência e largura de banda

Latência: tempo de ida e volta de uma mensagem (RTT).
Largura de banda: quantos dados passam por segundo. Em apps, as requisições são pequenas: a latência pesa mais.

Bloqueio na fila

No TCP, um pacote perdido segura todos os seguintes até ser reenviado, mesmo que pertençam a outras requisições.

TLS: como o HTTPS protege a conversa



TLS 1.3: uma ida e volta para combinar as chaves

As três garantias do TLS

Confidencialidade: ninguém no caminho lê. **Integridade**: ninguém altera sem ser notado. **Autenticidade**: o app fala mesmo com o servidor certo.

Duas criptografias juntas

A criptografia assimétrica (pares de chaves) é usada só para combinar uma chave secreta; os dados seguem com criptografia simétrica, muito mais rápida (AES ou ChaCha20).

Cadeia de confiança

O certificado do servidor é assinado por uma autoridade certificadora em que o sistema operacional já confia. Certificado vencido ou de outro domínio: conexão recusada.

A evolução do HTTP

Versão	Ano	Novidade	Efeito no app
HTTP/1.0	1996	uma conexão TCP por requisição	lento: aperto de mão a cada pedido
HTTP/1.1	1997	conexão reaproveitada (*keep-alive*)	menos apertos de mão, mas um pedido por vez em cada conexão
HTTP/2	2015	binário, vários pedidos simultâneos na mesma conexão, cabeçalhos comprimidos	muitas requisições pequenas ficam baratas
HTTP/3	2022	roda sobre QUIC (UDP), sem bloqueio na fila do TCP	a conexão sobrevive à troca do Wi-Fi para o 4G

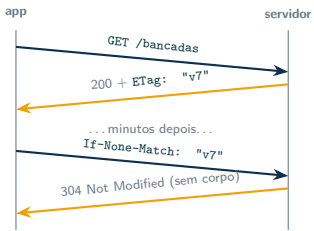
Um pouco de história

Tim Berners-Lee criou o HTTP e a Web no CERN, entre 1989 e 1991. Três décadas depois, o mesmo modelo de requisição e resposta move quase todos os apps.

Quem fala o quê

OkHttp usa HTTP/2; o Cronet, motor de rede do Chrome, também HTTP/3. No Flutter, `cronet_http` e `cupertino_http` usam as pilhas nativas do Android e do iOS.

Cache HTTP e requisições condicionais



ETag

Identificador da versão de um recurso. Na próxima vez, o app pergunta "mudou desde a v7?"; se não mudou, a resposta 304 vem vazia e o app usa a cópia que já tem.

`Cache-Control`	**Significado**
`max-age=60`	pode reutilizar por 60 s sem perguntar
`no-cache`	pode guardar, mas pergunte antes de usar
`no-store`	não guarde (dados sensíveis)

Dica

OkHttp e Dio têm cache HTTP: configurá-lo economiza dados móveis e bateria.

Serialização: de objetos a bytes

Serialização

Converter objetos em bytes para enviar ou guardar; desserializar é o caminho de volta. Respostas comprimidas (gzip, brotli) reduzem muito o JSON.

Formato	Características	Uso típico
JSON	texto legível, sem esquema obrigatório	APIs REST
Protocol Buffers	binário, com esquema, menor e mais rápido	gRPC, dados volumosos
CBOR, MessagePack	JSON em formato binário	IoT

Evolução sem quebrar

Apps antigos continuam instalados por meses. O servidor só acrescenta campos; o app ignora campos desconhecidos e tem valores padrão para os que faltarem. Mudanças incompatíveis vão para uma nova versão da API (`/v2`).

JSON grande trava a tela

Decodificar megabytes de JSON é trabalho de CPU: faça fora da *thread* da interface (Módulo 06).

Redes móveis: latência, energia e falhas

O rádio gasta mesmo parado

Depois de cada transferência, o rádio celular fica alguns segundos em modo de alta potência antes de voltar ao repouso. Várias requisições espalhadas gastam mais bateria que as mesmas requisições agrupadas.

Método	Repetir é seguro?
`GET`, `PUT`, `DELETE`	sim: são idempotentes (repetir dá o mesmo resultado)
`POST`	não: repetir pode criar duas reservas

Defesas contra falhas

Tempo limite em toda requisição; **novas tentativas** com espera exponencial e um pouco de aleatoriedade (*jitter*); **disjuntor** que para de insistir num servidor fora do ar.

"Conectado" não é "com internet"

Wi-Fi de hotel com página de login, rede fraca ou servidor fora do ar: a única prova de rede é a requisição que funcionou.

Tempo real: *polling*, SSE, WebSocket e *push*

Técnica	Como funciona	Custo	Quando usar
Polling	o app pergunta de tempos em tempos	alto: muitas requisições vazias	dados que mudam pouco
Long polling	o servidor segura a resposta até haver novidade	médio	quando nada melhor está disponível
SSE	o servidor envia eventos por uma resposta HTTP que não termina	baixo; só servidor → app	painéis e notícias ao vivo
WebSocket	canal aberto nos dois sentidos	baixo por mensagem; conexão aberta	chat, leituras ao vivo
MQTT	publicação e assinatura por tópicos	muito baixo	sensores e IoT (Módulo 13)
Push (FCM, APNs)	o sistema entrega mesmo com o app fechado	quase zero para o app	avisos e alertas (Módulo 12)

Uma conexão só, do sistema

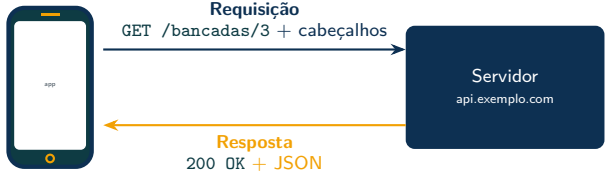
O *push* usa uma única conexão mantida pelo sistema para todos os apps: é o jeito mais econômico de receber avisos.

Atenção

Conexões abertas não sobrevivem ao segundo plano: o sistema as encerra. Para avisar um app fechado, só o *push*.

Fundamentos de HTTP

Requisição e resposta



Requisição

```
GET /v1/bancadas/3 HTTP/1.1
Host: api.labmonitor.ifma.edu.br
Accept: application/json
Authorization: Bearer eyJhbGciOi...
```

Resposta

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "id": 3, "nome": "Bancada 3",
  "temp_c": 24.8, "livre": true }
```

Métodos e códigos de status

Método	Semântica
`GET`	ler um recurso (sem corpo)
`POST`	criar um recurso ou disparar ação
`PUT`	substituir um recurso inteiro
`PATCH`	alterar parte de um recurso
`DELETE`	remover um recurso

Idempotência

`GET`, `PUT` e `DELETE` podem ser repetidos sem efeito extra, por isso é seguro tentar de novo após uma falha. `POST`, em geral, não.

Faixa	Significado e exemplos
2xx	sucesso: `200 OK`, `201 Created`, `204 No Content`
3xx	redirecionamento: `301`, `304 Not Modified`
4xx	erro do cliente: 400 inválido, 401 sem login, 403 sem permissão, 404 não existe, 409 conflito, 429 limite de requisições
5xx	erro do servidor: 500, 502, 503 indisponível

Dica

4xx: o app precisa corrigir algo (login, dados). 5xx e falhas de conexão: vale tentar de novo mais tarde.

APIs REST e JSON

Projetando a API do LabMonitor

Método	Endpoint	Ação
`GET`	`/v1/bancadas`	lista todas as bancadas
`GET`	`/v1/bancadas/{id}`	detalha uma bancada
`GET`	`/v1/bancadas/{id}/leituras?desde=...`	leituras a partir de uma data (query string)
`POST`	`/v1/reservas`	cria uma reserva (corpo JSON)
`DELETE`	`/v1/reservas/{id}`	cancela uma reserva

REST em uma frase

URLs identificam **recursos** (substantivos), métodos HTTP dizem **o que fazer** (verbos), e o estado trafega em JSON.

API pública para praticar

A **Open-Meteo** não exige chave:

`https://api.open-meteo.com/v1/forecast?latitude=-2.53&longitude=-44.30&current=temperature_2m`

devolve a temperatura atual em São Luís.

Configuração no Android

Permissão e HTTPS

AndroidManifest.xml

```
<uses-permission
  android:name="android.permission.INTERNET" />

<application
  android:networkSecurityConfig=
    "@xml/network_security_config" ... >
```

res/xml/network_security_config.xml

```
<network-security-config>
  <!-- HTTP liberado SÓ para o ESP32 na rede local -->
  <domain-config cleartextTrafficPermitted="true">
    <domain
      includeSubdomains="false">192.168.4.1</domain>
  </domain-config>
</network-security-config>
```

HTTPS por padrão

Desde o Android 9, o tráfego em texto puro (`http://`) é bloqueado. Use `https://` sempre que possível.

Dispositivos IoT

Placas como o ESP32 muitas vezes servem HTTP simples na rede local. Libere **apenas aquele endereço**, nunca `cleartextTrafficPermitted="true"` global.

Dica

No emulador, `10.0.2.2` aponta para o `localhost` do seu computador.

Retrofit e kotlinx.serialization

Modelando o JSON

Resposta da Open-Meteo (resumo)

```
{
  "latitude": -2.53,
  "longitude": -44.3,
  "current": {
    "time": "2026-11-01T15:00",
    "temperature_2m": 30.4,
    "relative_humidity_2m": 62
  }
}
```

Dica

Ferramentas como o *JSON to Kotlin Class* (plugin da IDE) geram os DTOs a partir de um exemplo.

ClimaDto.kt

```kotlin
@Serializable
data class ClimaDto(
  val latitude: Double,
  val longitude: Double,
  val current: AtualDto
)

@Serializable
data class AtualDto(
  val time: String,
  @SerialName("temperature_2m")
  val temperatura: Double,
  @SerialName("relative_humidity_2m")
  val umidade: Int? = null      // opcional
)
```

Conceito

`@SerialName` liga o nome do JSON ao nome Kotlin. Valores padrão tornam campos opcionais.

A interface da API

LabApi.kt

```
interface LabApi {
  @GET("v1/bancadas")
  suspend fun listarBancadas(): List<BancadaDto>

  @GET("v1/bancadas/{id}")
  suspend fun bancada(@Path("id") id: Int): BancadaDto

  @GET("v1/bancadas/{id}/leituras")
  suspend fun leituras(
    @Path("id") id: Int,
    @Query("desde") desde: String
  ): List<LeituraDto>

  @POST("v1/reservas")
  suspend fun reservar(@Body r: NovaReservaDto): ReservaDto

  @DELETE("v1/reservas/{id}")
  suspend fun cancelar(@Path("id") id: Long): Response<Unit>
}
```

Anotação	Função
`@GET`, `@POST`...	método e caminho
`@Path`	substitui `{id}` na URL
`@Query`	`?chave=valor`
`@Body`	objeto serializado no corpo
`@Header`	cabeçalho específico

Funções suspend

O Retrofit executa a chamada fora da thread principal e devolve o objeto já convertido. Status fora de 2xx lança `HttpException`.

Dica

Use `Response<T>` quando precisar ler o código de status ou os cabeçalhos.

Montando o cliente com OkHttp e Hilt

di/NetworkModule.kt

```kotlin
@Module @InstallIn(SingletonComponent::class)
object NetworkModule {
  @Provides @Singleton
  fun okHttp(sessao: SessaoRepository) = OkHttpClient.Builder()
    .connectTimeout(10, TimeUnit.SECONDS)
    .addInterceptor { chain ->          // token em toda requisição
      chain.proceed(chain.request().newBuilder()
        .header("Authorization", "Bearer ${sessao.token()}")
        .build())
    }
    .addInterceptor(HttpLoggingInterceptor().apply {
      level = if (BuildConfig.DEBUG) Level.BODY else Level.NONE
    }).build()
  @Provides @Singleton
  fun api(client: OkHttpClient): LabApi = Retrofit.Builder()
    .baseUrl("https://api.labmonitor.ifma.edu.br/")
    .client(client)
    .addConverterFactory(Json { ignoreUnknownKeys = true }
      .asConverterFactory("application/json".toMediaType()))
    .build().create(LabApi::class.java)
}
```

Camadas

Retrofit transforma a interface em chamadas HTTP; **OkHttp** faz a conexão (cache, timeouts, interceptadores); **kotlinx.serialization** converte JSON.

`ignoreUnknownKeys`

Se a API ganhar campos novos, o app antigo continua funcionando.

Atenção

Log de corpo (`BODY`) só em debug: ele mostra tokens e dados pessoais.

Erros e estados

O que pode dar errado

Exceção	Causa provável
`UnknownHostException`	sem internet / DNS
`SocketTimeoutException`	servidor lento
`HttpException` (4xx/5xx)	resposta de erro
`SerializationException`	JSON diferente do esperado
`SSLHandshakeException`	certificado inválido

Atenção

Todas as de rede herdam de `IOException`. Trate-as **no repositório** e entregue à UI um erro com significado.

Erros com significado

```
sealed interface ErroRede {
  data object SemConexao : ErroRede
  data object NaoAutorizado : ErroRede
  data class Servidor(val codigo: Int) : ErroRede
  data object Desconhecido : ErroRede
}

suspend fun <T> chamarApi(
  bloco: suspend () -> T
): Result<T> = try {
  Result.success(bloco())
} catch (e: CancellationException) { throw e
} catch (e: IOException) {
  Result.failure(FalhaRede(ErroRede.SemConexao))
} catch (e: HttpException) {
  Result.failure(FalhaRede(when (e.code()) {
    401, 403 -> ErroRede.NaoAutorizado
    else -> ErroRede.Servidor(e.code()) }))
}
```

Do erro à tela

ClimaViewModel.kt

```
fun carregar() = viewModelScope.launch {
  _estado.update { it.copy(carregando = true, erro = null) }
  chamarApi { api.clima(-2.53, -44.30) }
    .onSuccess { dto ->
      _estado.update {
        it.copy(carregando = false,
          temperatura = dto.current.temperatura)
      }
    }
    .onFailure { e ->
      _estado.update {
        it.copy(carregando = false,
          erro = (e as? FalhaRede)?.tipo?.mensagem()
            ?: "Erro inesperado")
      }
    }
}
```

Carregando: indicador de progresso

Sucesso: dados na tela

Vazio: mensagem + ação

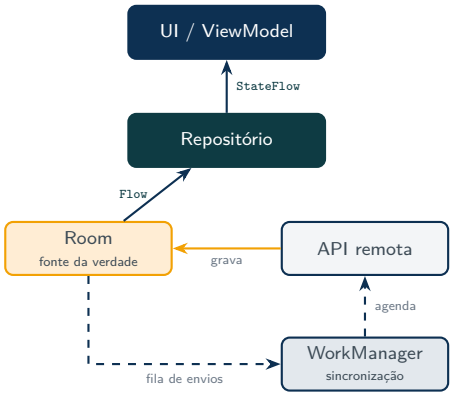
Erro: explicação + “Tentar de novo”

Repetir com espera crescente

Para falhas temporárias, tente de novo após 1 s, 2 s, 4 s... (*exponential backoff*). O operador `retryWhen` do Flow e o WorkManager fazem isso.

Offline-first

Estratégia offline-first



Leitura

A UI lê **sempre** do banco local. A rede apenas atualiza o banco, e a tela reage sozinha.

Escrita

Grave localmente na hora (marcado como "pendente") e envie ao servidor quando houver conexão, com WorkManager (Módulo 13).

Resultado

O app abre instantaneamente, funciona no laboratório sem Wi-Fi e economiza dados móveis.

Implementando no repositório

BancadaRepositoryImpl.kt

```
override fun bancadas(): Flow<List<Bancada>> =
  dao.observarTodas()
    .map { lista -> lista.map { it.paraModelo() } }
    .onStart { atualizarSeAntigo() }   // dispara refresh

private suspend fun atualizarSeAntigo() {
  if (agora() - ajustes.ultimaSync() < 5.minutes) return
  chamarApi { api.listarBancadas() }.onSuccess { dtos ->
    dao.substituirTodas(dtos.map { it.paraEntidade() })
    ajustes.registrarSync(agora())
  } // em falha: a UI segue com os dados em cache
}

override suspend fun reservar(r: NovaReserva) {
  dao.inserirReserva(r.paraEntidade(pendente = true))
  agendador.agendarEnvioPendentes()   // WorkManager
}
```

Conflitos

Reservas offline conflitantes: o servidor decide. Trate `409 Conflict` avisando o usuário.

Mostre a idade dos dados

"Atualizado há 3 min" e um ícone de offline deixam claro o que se vê.

Dica

Monitore a conectividade com o `ConnectivityManager` (`registerDefaultNetworkCallback`) convertido em `Flow` via `callbackFlow`.

Imagens, alternativas e segurança

Imagens com Coil e alternativas ao Retrofit

Coil em Compose

```
AsyncImage(
  model = ImageRequest.Builder(LocalContext.current)
    .data(equipamento.fotoUrl)
    .crossfade(true)
    .build(),
  placeholder = painterResource(R.drawable.carregando),
  error = painterResource(R.drawable.sem_foto),
  contentDescription = equipamento.descricao,
  contentScale = ContentScale.Crop,
  modifier = Modifier.size(72.dp)
    .clip(RoundedCornerShape(8.dp))
)
```

Coil

Baixa, redimensiona e guarda imagens em cache (memória e disco) automaticamente.

Biblioteca	Quando considerar
Retrofit	padrão de mercado em apps Android
Ktor Client	projetos Kotlin Multiplatform (Módulo 19)
OkHttp puro	chamadas avulsas, download de arquivos
WebSocket (OkHttp/Ktor)	dados em tempo real bidirecionais
MQTT	telemetria IoT (Módulo 13)

Dica

A arquitetura em camadas permite trocar Retrofit por Ktor mexendo só na fonte remota.

Segurança na comunicação

Nunca coloque segredos no app

Tudo o que vai no APK pode ser extraído: chaves de API, senhas de banco, tokens de serviços pagos. **Ofuscação não protege segredos.**

Solução: backend intermediário

O app chama **o seu servidor** (ou uma Cloud Function), que guarda a chave e chama o serviço externo. O app se autentica com o token do usuário.



Checklist

- HTTPS em tudo.
- Tokens de curta duração, com renovação.
- Valide tudo também no servidor.
- Limite de requisições no backend.
- Logs de rede desligados em release.

Fechamento

Resumo

Resumo

- HTTP = requisição (método, URL, cabeçalhos, corpo) + resposta (status, cabeçalhos, corpo).
- DTOs `@Serializable` espelham o JSON; Retrofit transforma a interface em chamadas `suspend`.
- OkHttp cuida de timeouts, interceptadores (token, log) e cache.
- Converta exceções em erros com significado no repositório e mostre estados claros na UI.
- Offline-first: a UI lê do Room, a rede atualiza o banco e as escritas entram numa fila.
- Nada de segredos no app: use um backend intermediário e HTTPS.

Glossário (1/2)

Termo	Significado
Protocolo	regras de formato e de ordem das mensagens entre computadores
TCP/IP	pilha de protocolos da internet: aplicação, transporte, internet e enlace
DNS	sistema que traduz nomes de domínio em endereços IP
NAT	tradução de endereços que esconde o celular atrás de um IP público
TCP	transporte com conexão, entrega garantida e ordem
UDP	transporte sem conexão nem garantias, de baixa latência
TLS	protocolo que cifra e autentica a conexão (o “S” do HTTPS)
Latência (RTT)	tempo de ida e volta de uma mensagem
HTTP/2 e HTTP/3	vários pedidos na mesma conexão; o HTTP/3 roda sobre QUIC (UDP)

Glossário (2/2)

Termo	Significado
REST	estilo de API com recursos em URLs e métodos HTTP
ETag	identificador de versão que permite a resposta 304 sem corpo
Serialização	conversão de objetos em bytes (JSON, Protocol Buffers)
Idempotência	repetir a operação produz o mesmo resultado
Retrofit	cliente HTTP declarativo: a API vira uma interface Kotlin
OkHttp	motor HTTP do Retrofit, com interceptadores e cache
Interceptador	código que examina ou altera toda requisição e resposta
WebSocket	canal aberto nos dois sentidos para tempo real

Questões de revisão

1. Descreva, camada por camada, o que acontece quando o app chama `GET https://api.labmonitor.dev/v1/bancadas`.
2. Por que o servidor não consegue abrir uma conexão até o celular? Que soluções existem?
3. Por que, em apps, a latência costuma pesar mais que a largura de banda?
4. Que três garantias o TLS oferece e como o app sabe que fala com o servidor certo?
5. O que o HTTP/3 traz de especialmente útil para um celular em movimento?
6. Explique como *ETag* e a resposta 304 economizam dados.
7. Que cuidados permitem mudar a API sem quebrar versões antigas do app?
8. Uma requisição `POST /reservas` deu tempo limite esgotado. É seguro repetir? Como tornar seguro?

Exercícios

Exercício 1 — Open-Meteo

Crie uma tela que mostra temperatura e umidade atuais de São Luís usando a Open-Meteo, com os estados carregando, sucesso e erro, e um botão "Atualizar".

Exercício 2 — Falhas

Teste o app em modo avião, com *throttling* de rede no emulador e com uma URL inexistente. Cada caso mostra uma mensagem adequada?

Exercício 3 — ESP32 local

Programe um ESP32 para servir `GET /leitura` com JSON na rede local. Consuma-o no app com Retrofit, liberando HTTP só para o IP da placa.

LabMonitor — Etapa 11

Adicione a fonte remota (API real ou um servidor *mock*) ao repositório de bancadas com estratégia offline-first, mostrando "atualizado há X min" no painel.

Para aprofundar

- Conectar à rede:
  `https://developer.android.com/develop/connectivity/network-ops/connecting`
- Network security config:
  `https://developer.android.com/privacy-and-security/security-config`
- Retrofit: `https://square.github.io/retrofit/`
- kotlinx.serialization: `https://github.com/Kotlin/kotlinx.serialization`
- Apps offline-first:
  `https://developer.android.com/topic/architecture/data-layer/offline-first`
- Coil: `https://coil-kt.github.io/coil/` Open-Meteo: `https://open-meteo.com`

Respostas das questões de revisão

1. O DNS traduz o nome em IP; o TCP abre a conexão na porta 443; o TLS combina as chaves e confere o certificado; o HTTP envia a requisição cifrada; tudo viaja em pacotes IP por Wi-Fi ou rede móvel.
2. O celular fica atrás de NAT, com endereço privado, e não aceita conexões novas. Soluções: notificações *push* (FCM, APNs) ou conexões abertas pelo próprio app (WebSocket, MQTT).
3. As requisições são pequenas: o tempo de ida e volta (aperto de mão, DNS, TLS) domina o tempo de transferência.
4. Confidencialidade, integridade e autenticidade. O certificado do servidor é assinado por uma autoridade em que o sistema confia e precisa valer para aquele domínio.
5. Roda sobre QUIC: sem bloqueio na fila do TCP e com conexão que sobrevive à troca de rede, como do Wi-Fi para o 4G.
6. O app envia a versão que tem (`If-None-Match`); se nada mudou, o servidor responde 304 sem corpo e o app reutiliza a cópia local.
7. Só acrescentar campos, ignorar campos desconhecidos (`ignoreUnknownKeys`), ter valores padrão e criar `/v2` para mudanças incompatíveis.
8. Não, pois `POST` não é idempotente e pode criar duas reservas. Envie uma chave de idempotência, para o servidor reconhecer a repetição.

Fim do Módulo 11

Próximo: Módulo 12 — Firebase: autenticação, Firestore, regras de segurança, armazenamento e notificações push.